20/11/2022

Microprocessors

```
module foo (x, y, count, a, b);
input logic [4:0] x, count; // x and count 5-bit
input logic y;
output logic [2:0] a;
output logic [1:0] b;
assign diff = x | count
logic [15:0] mem[0:512] // 1KByte memory of 16-bit words
mem[65] //65th location, an 16-bit variable
```

1) declare input-output
2) declare local var
3) compute
4) endmodule

```
always_ff @(posedge clk)
begin
  X <= y } order is
  Y <= x } not important
end
```

```
always_comb
begin
  X = y } order is important
  Y = x }
end
```

no loop

`5'b 00101` → 5-bit binary number `-7'b 101` → true's comp. negative of 101

`11'hf7a` → the rightmost 11 bits of the hex number f7a

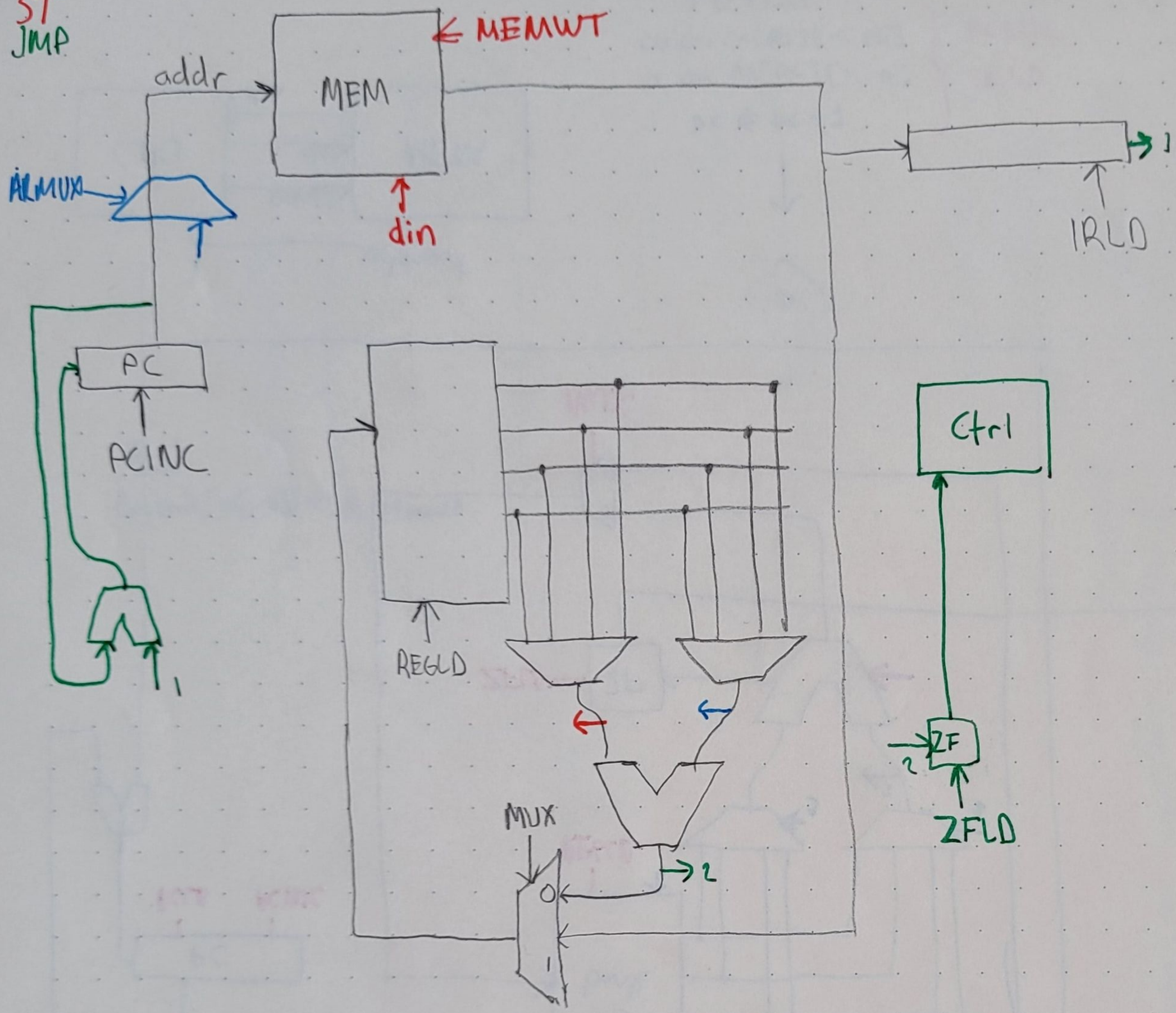
```
LDI
regbank[ir[2:0]] <= data-in
PC <= PC +1
State <= FETCH
```

```
ALU
regbank[ir[2:0]] <= result
zeroflag <= zeroresult
State <= FETCH
```

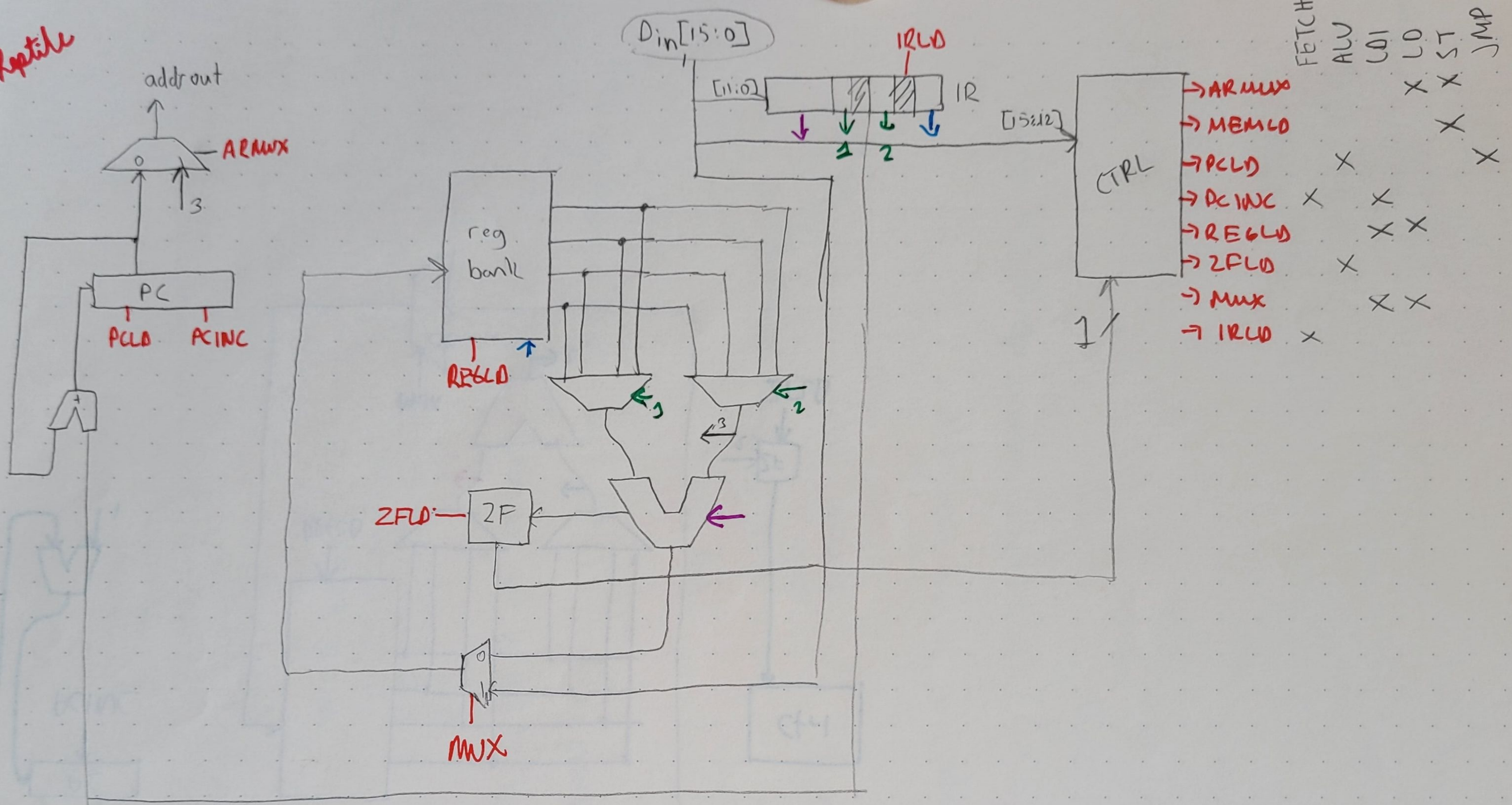
```
LD
regbank[ir[2:0]] <= data-in
State <= FETCH
ST
State <= FETCH
JMP
PC <= PC+ir
State <= FETCH
```

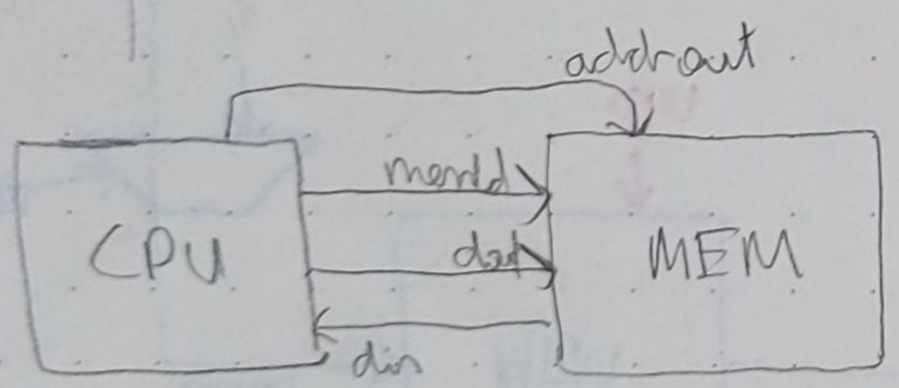
```
always_comb
 case(state)
   LD: address = regbank[ir[5:3]][11:0];
   ST: address = regbank[ir[5:3]][11:0];
   default: address = PC;
```

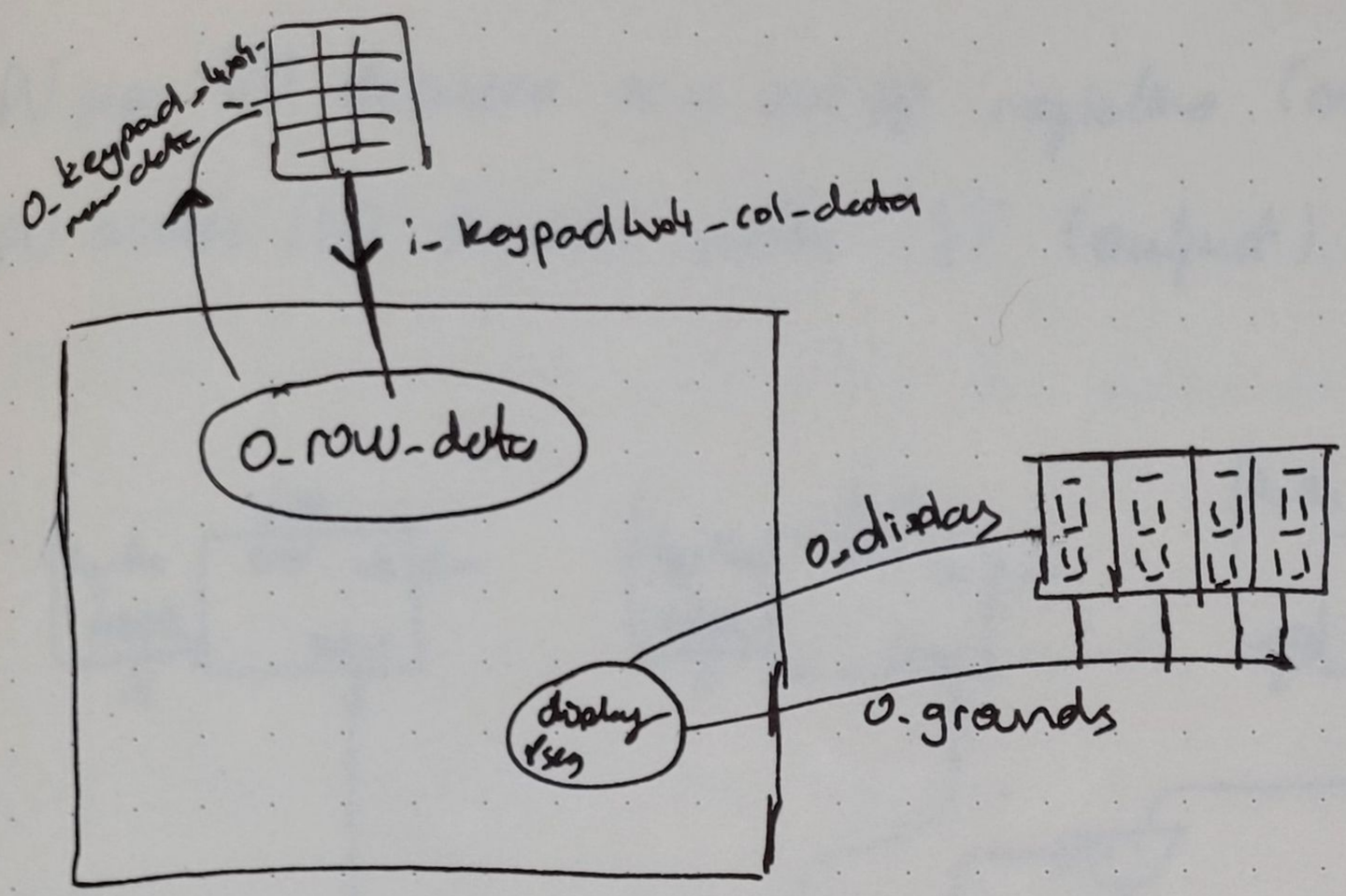
LD
ST
JMP

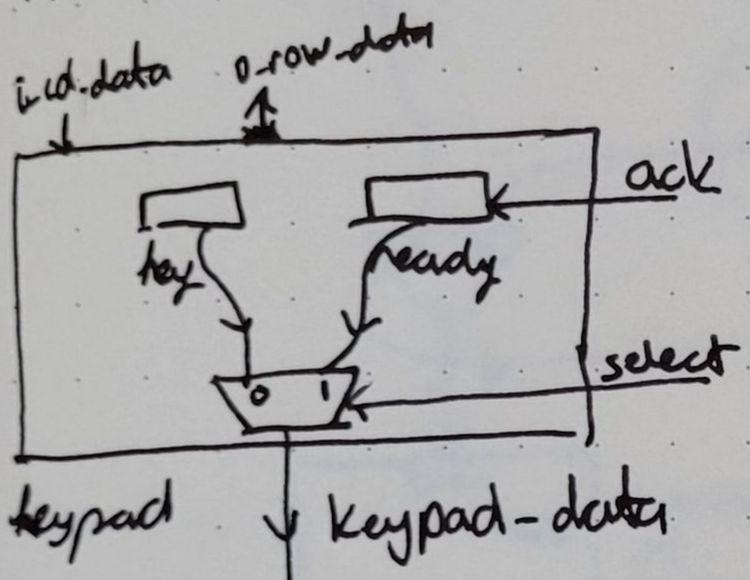


Reptile

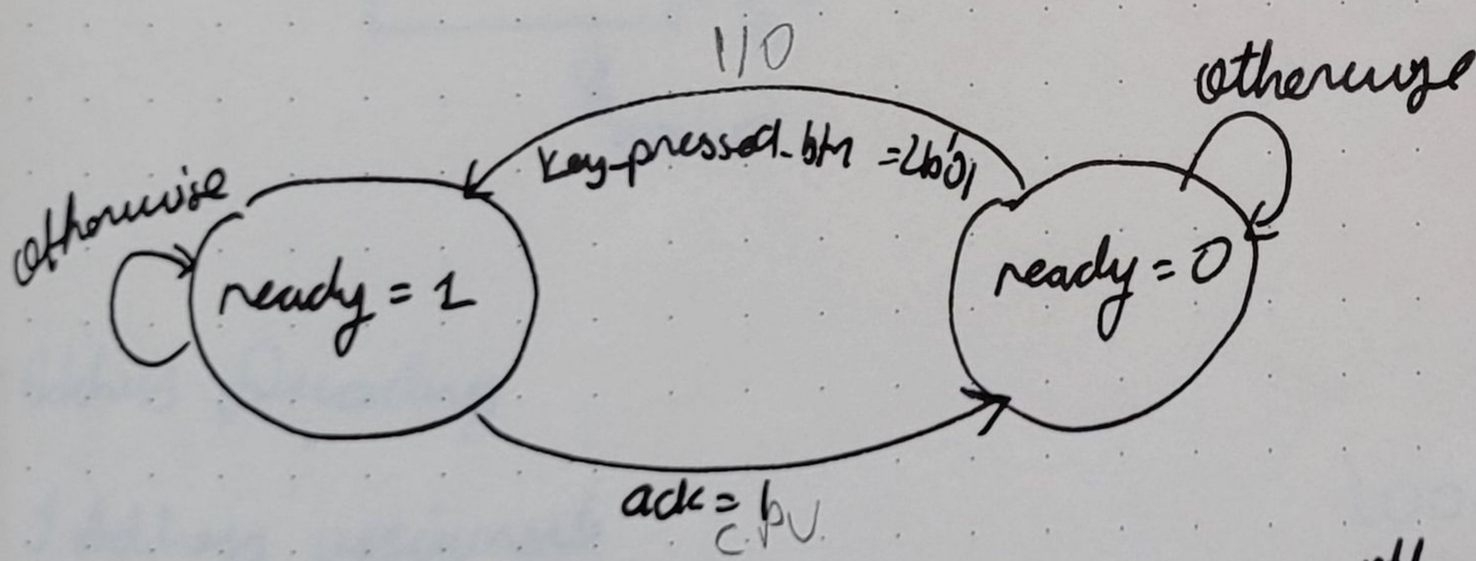

addrout
ARMUX
PC
PCLD
PCINC
reg bank
REGLD
Din[15:0]
IRLD
[1:0]
IR
[5:12]
CTRL
ARMUX
MEMLD
PCLD
PCINC
REGLD
ZFLD
MUX
IRLD
FETCH
ALU
LDI
LD
ST
JMP
ZFLD
ZF
MUX

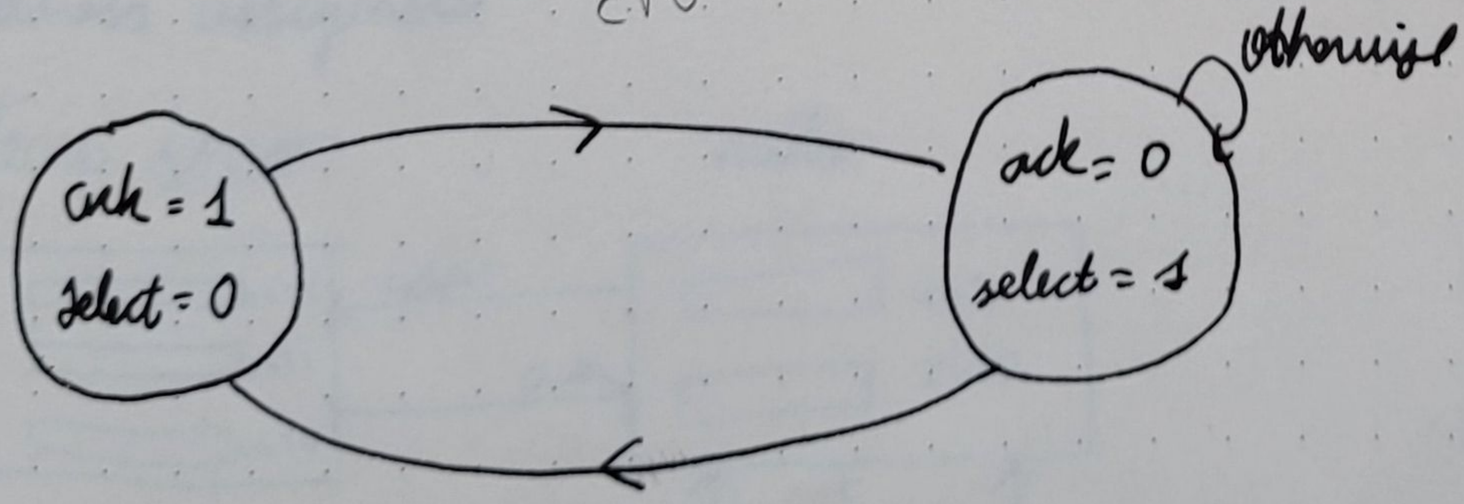

addrout
CPU
MEM
din





if ready register is zero key is not pressed





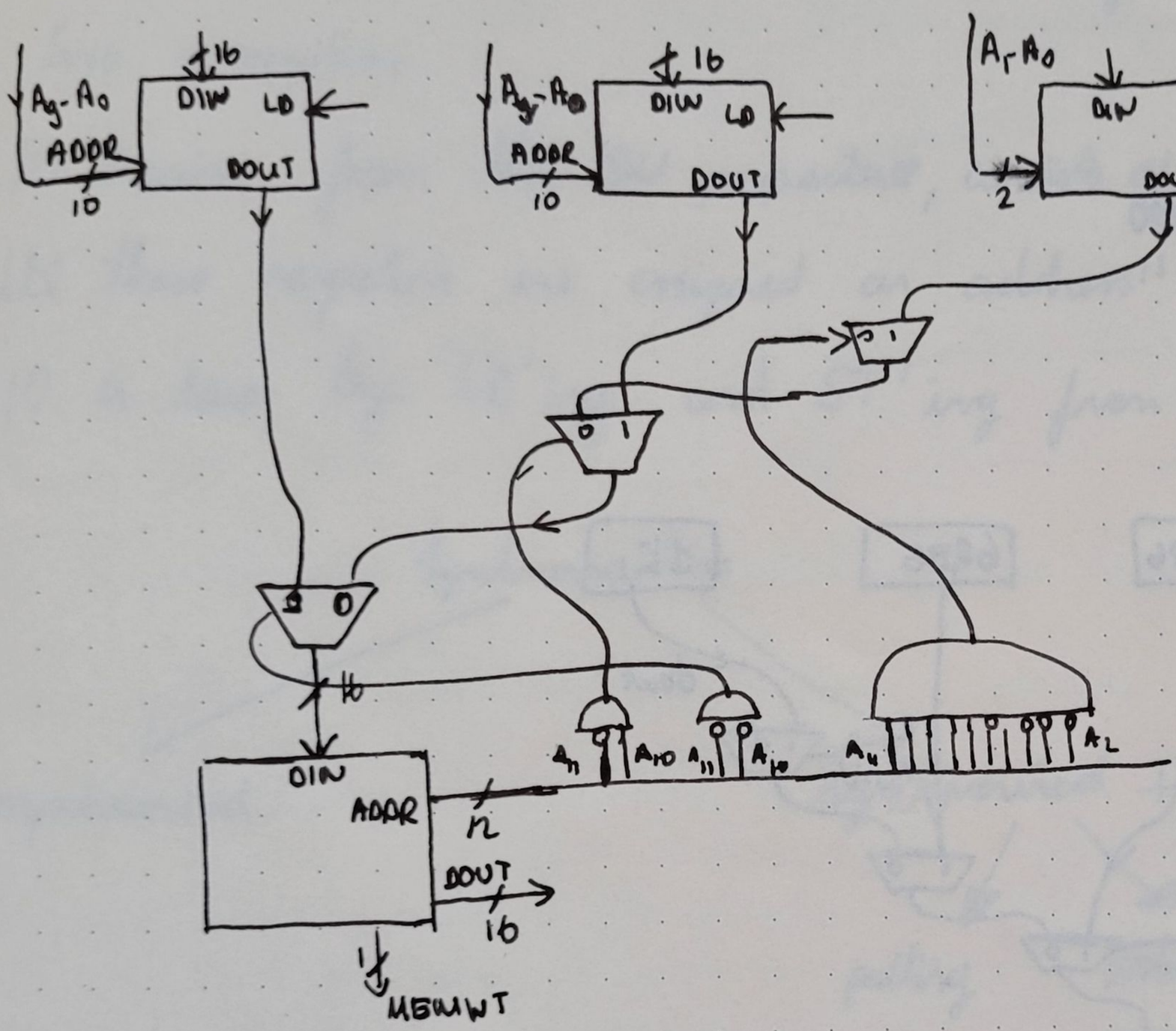
keypad_data[0] = 1

CPU sees I/O devices as a set of registers (memory)

CPU access I/O devices with ST (output) LD (input) instructions

$k \leq 2^m$

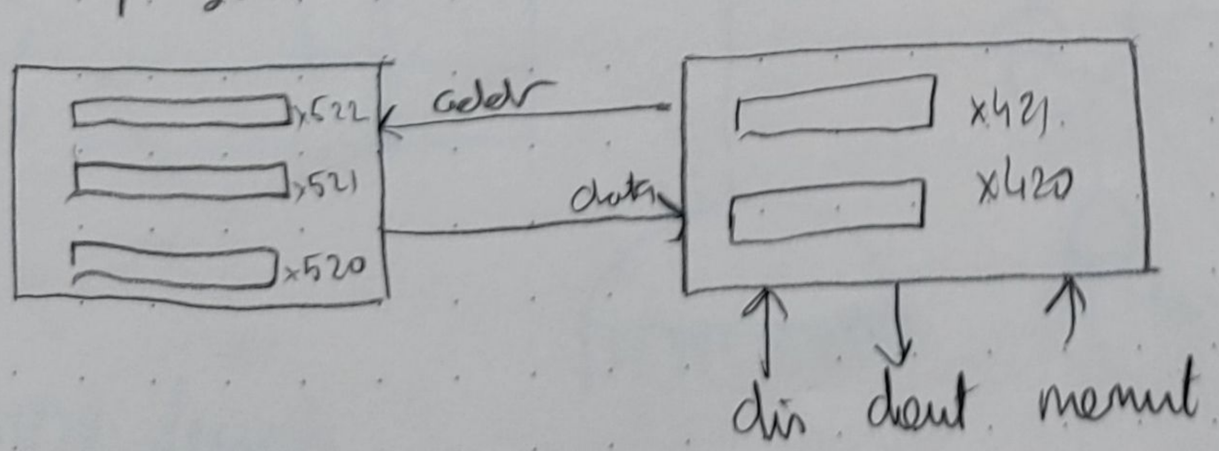
$q \cdot 2^m$



14/11/2022

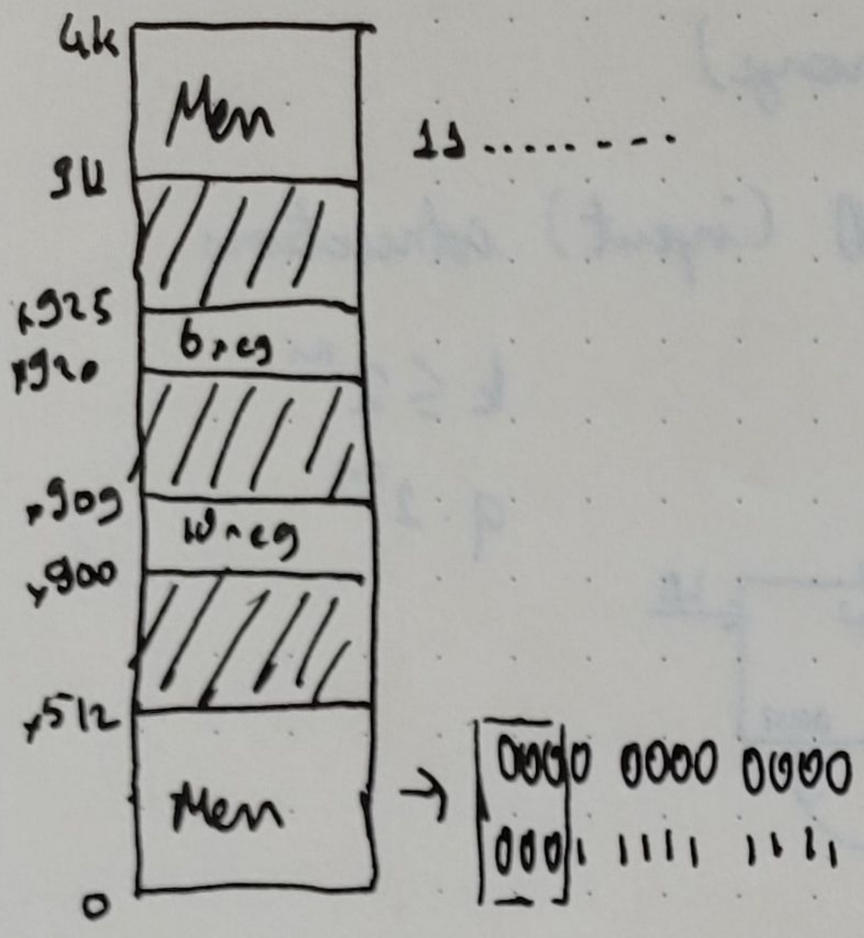
- Address Decoding

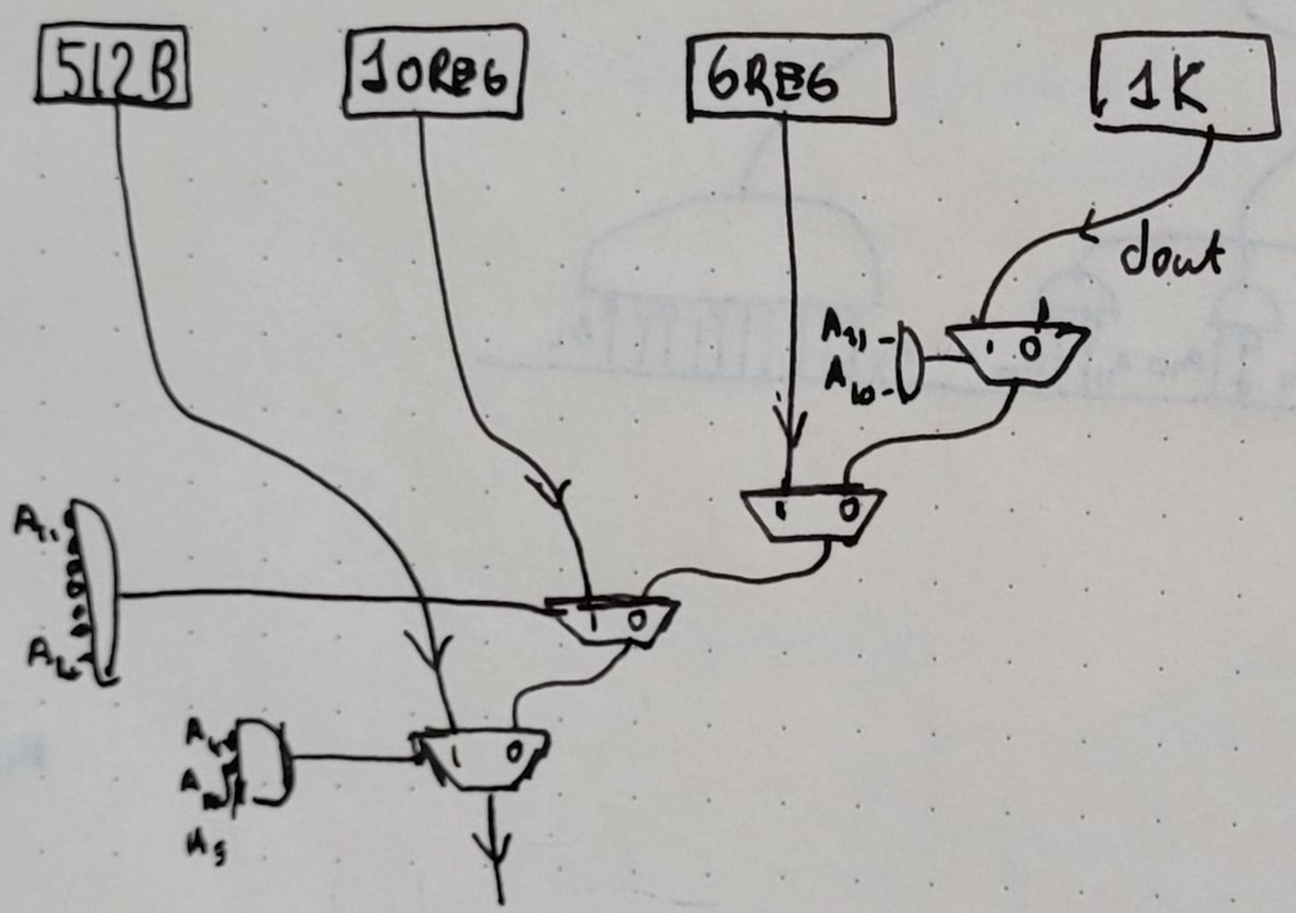
1. Address assignment




```
LOOP: LDI 0, 0x522
      LD  0, 0
      LDI 1, 0x421
      ST  1, 0
      JMP LOOP
```

14/11/2012





- Read Keypad

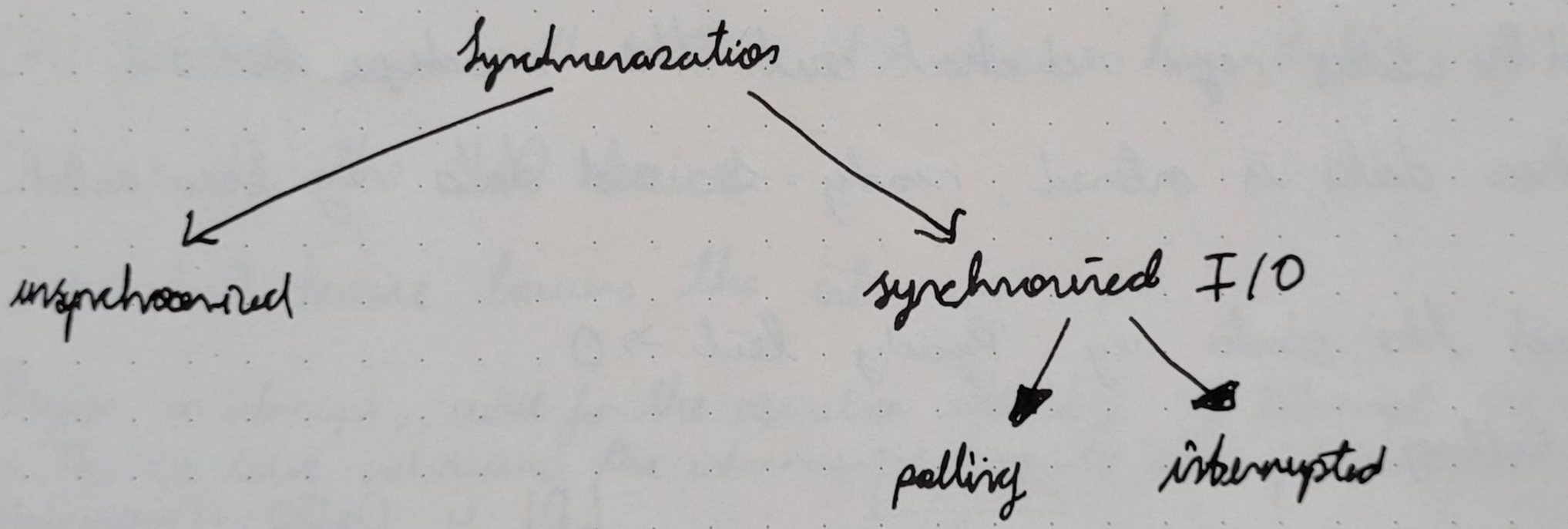
```
READ  LDI   0    0x801
      LD    0    0
      LDI   1    0x1
      AND   2    0     1
      JZ    READ
      LDI   0    0x300
      LD    0    0
      LDI   1    0x1400
      ST    1    0
      JMP   READ
```

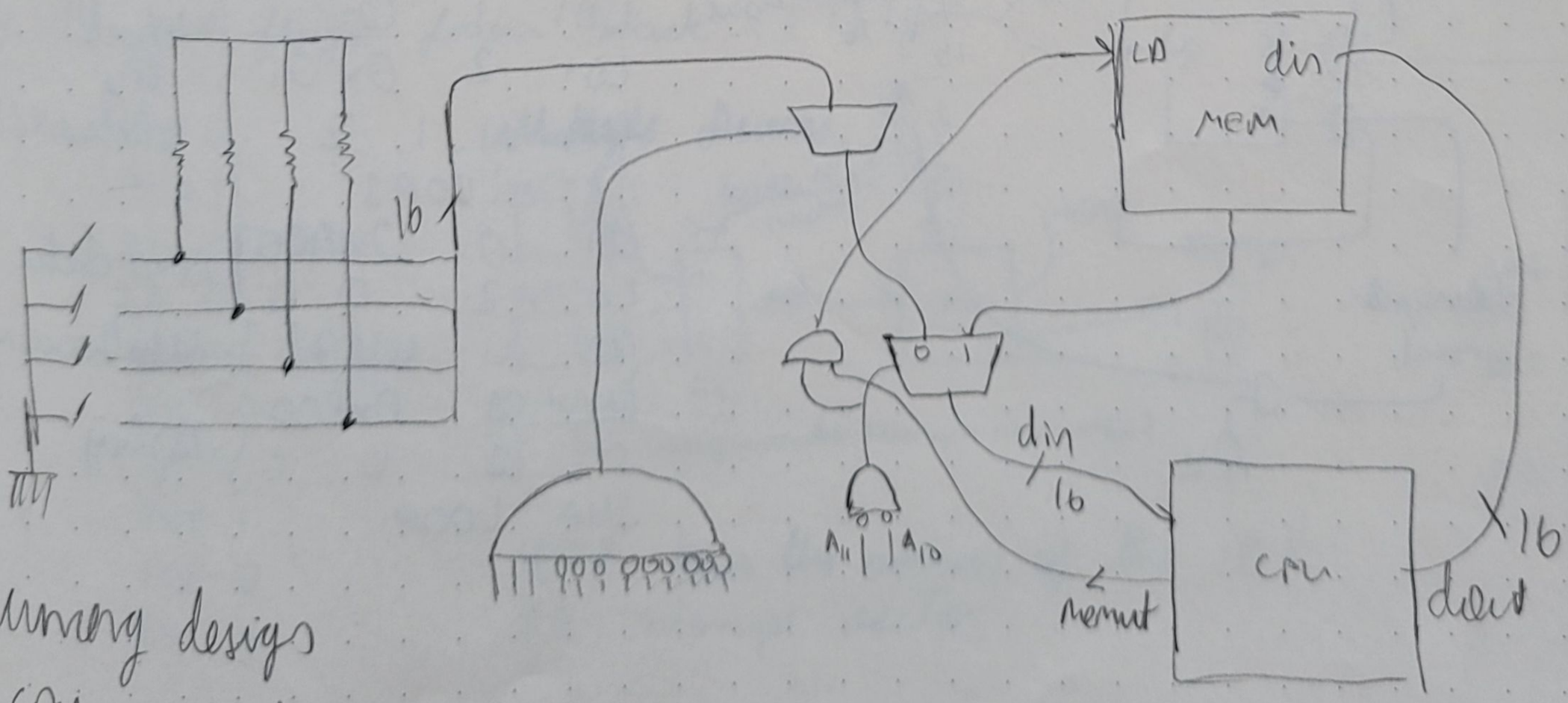
17/11/2022

Polling

In order to connect the keypad to the CPU properly, we need polling (note that we haven't connected the ACK pin yet)

- Our basic assumptions
- I/O devices from the SW perspective, consists of many registers
- All these registers are assigned an address
- I/O is done by LD'ing and ST'ing from these addresses

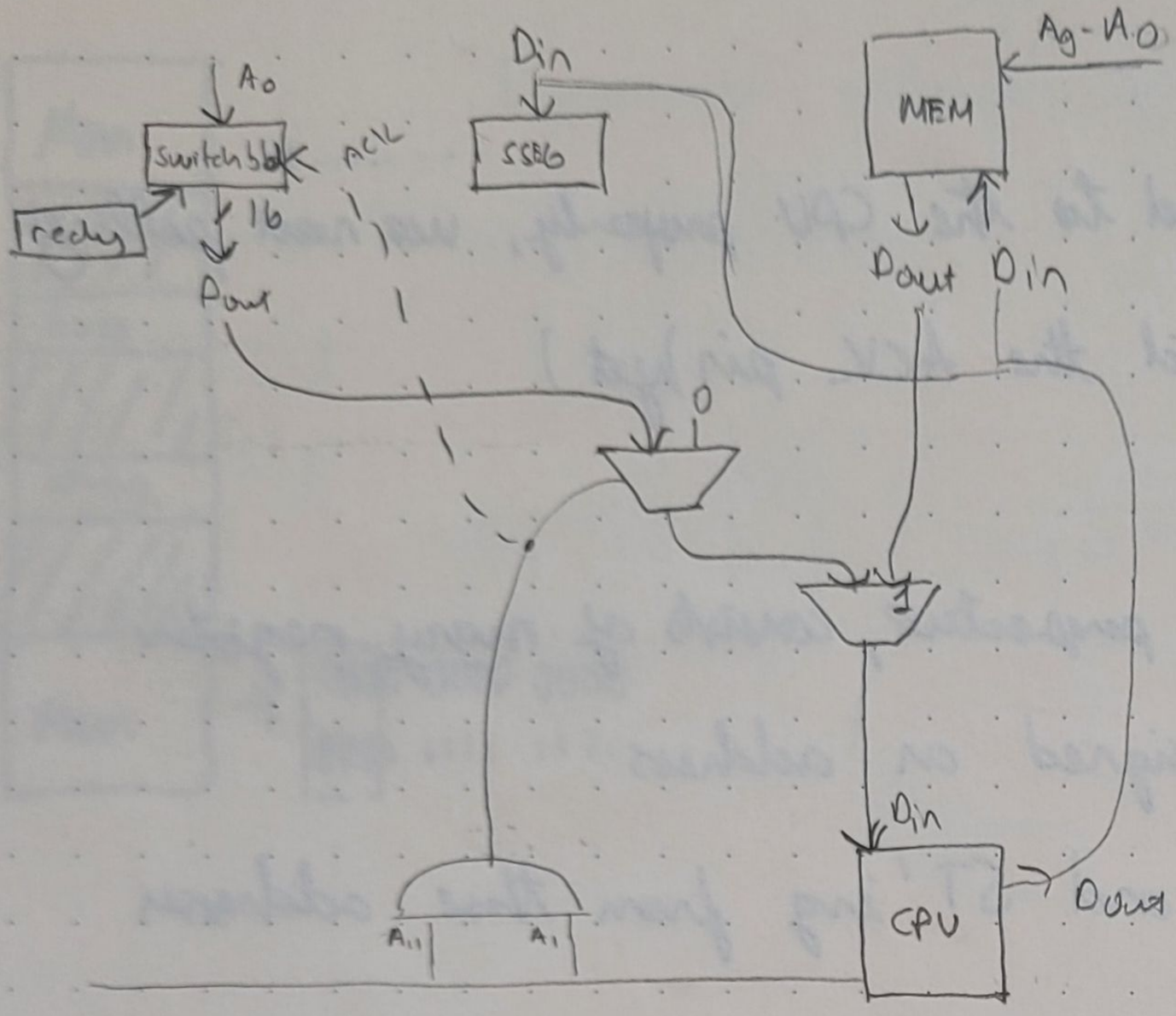


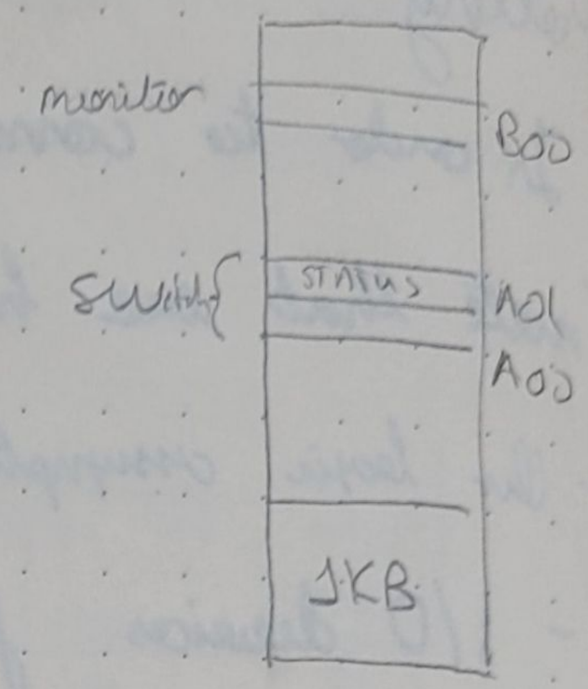


Wrong design

CPU may read before switch

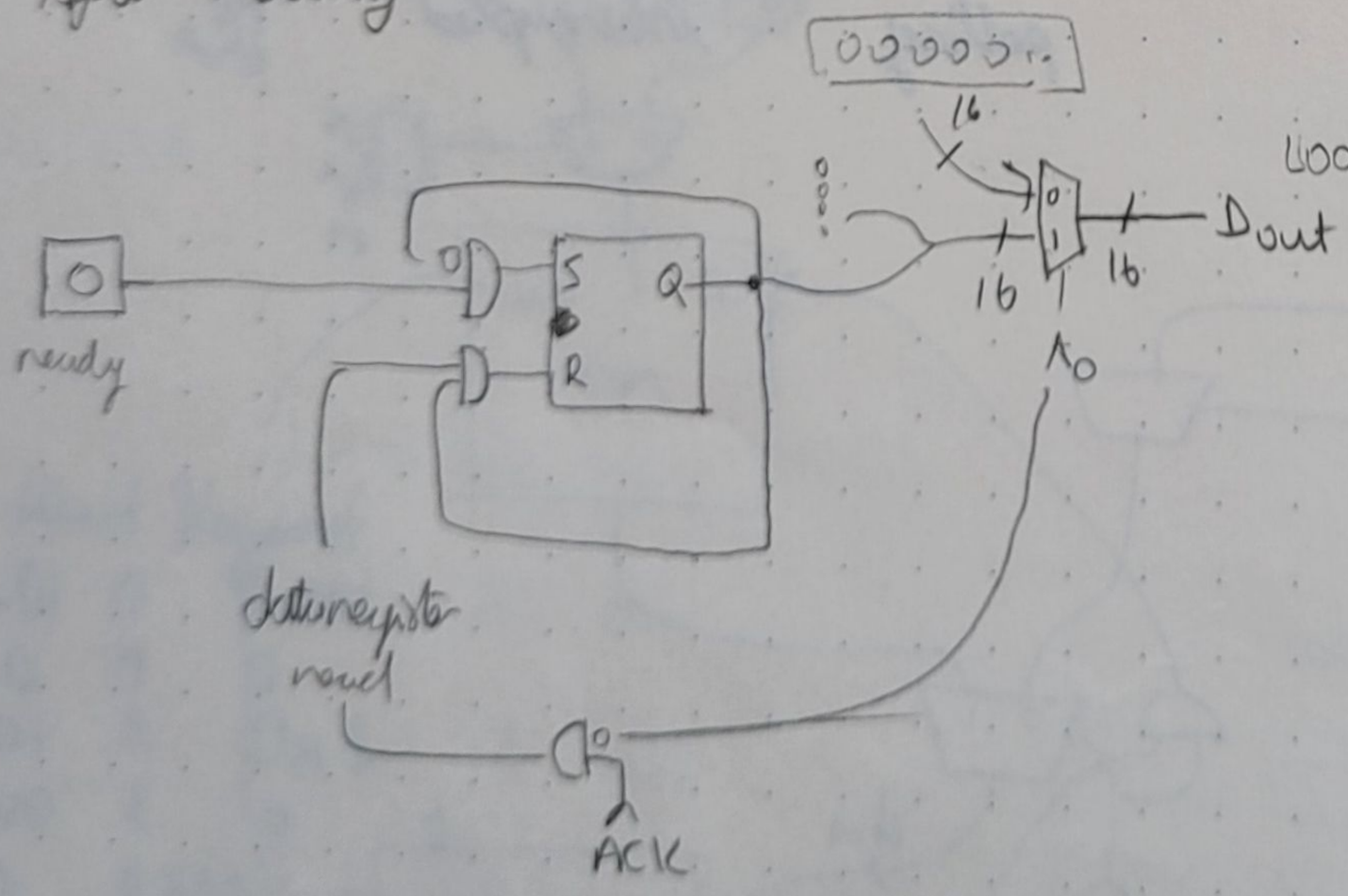
01/12/2022





1. Read the status reg. and check out the ready bit
2. When data is entered, ready = 1 and data reg have valid data
3. Read the data reg. Ready bit ⇒ 0

- After Polling




```
      LDI  4   0x00   } accumulator
LOOP1:LDI  0   0xA01  }
      LDI  1   0      }
      LDI  2   0x01   } check the status
      AND  1   1   2  }
      JZ   LOOP1
      LDI  0   0xA00  } read data
      LD   2   0      }
      ADD  4   4   2  } add to the acc. result
      LDI  1   0xB00  } display
      ST   1   4      }
      JMP  LOOP1
```

POP2
SP+=2

Stack

- Push
  - Copy from register to memory, decrement stack pointer by one
- Pop
  Increment stack pointer, copy from memory to register

12/12/2022

Interrupts

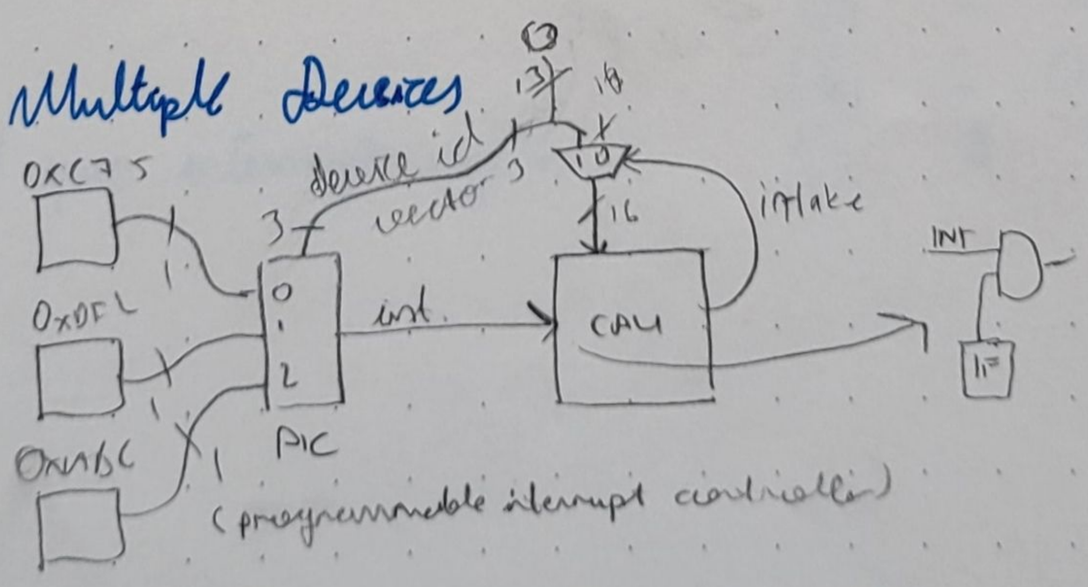
- I/O device obtains some data and puts the data to it's data reg
- I/O device raises interrupt pin
- CPU becomes aware that the I/O device has data
- CPU reads the I/O device
- The I/O device lowers the interrupt pin

Receive an interrupt, wait for the instruction boundry, if interrupts are enabled save the context (PC+FLAGS) and receive the interrupt, jump to interrupt service routine (ISR)

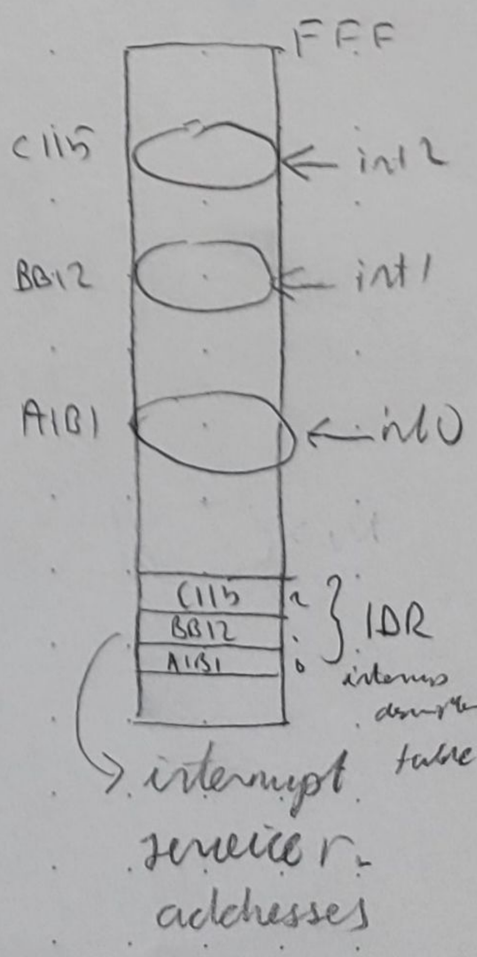
Read device and do necessary things, return from interrupt pop the contents of PC and flags from stack. (RTI in)

```
ISR  PUSH 0
     PUSH 1
     LD 0xF1A3
     LD 0, 0
     LDI 1, 0xABC1
     INC 1
     ST 1, 0
     POP 1
     POP 0
     RET 1
      ⋮
     DEC 3
     JZ LOOP
```

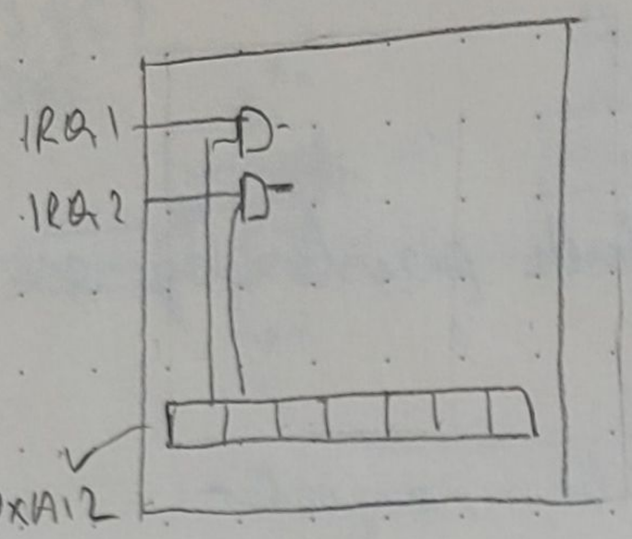
Multiple Devices



IDTR: keeps the address of the IDR
IRQ: interrupt vector



22/12/2022

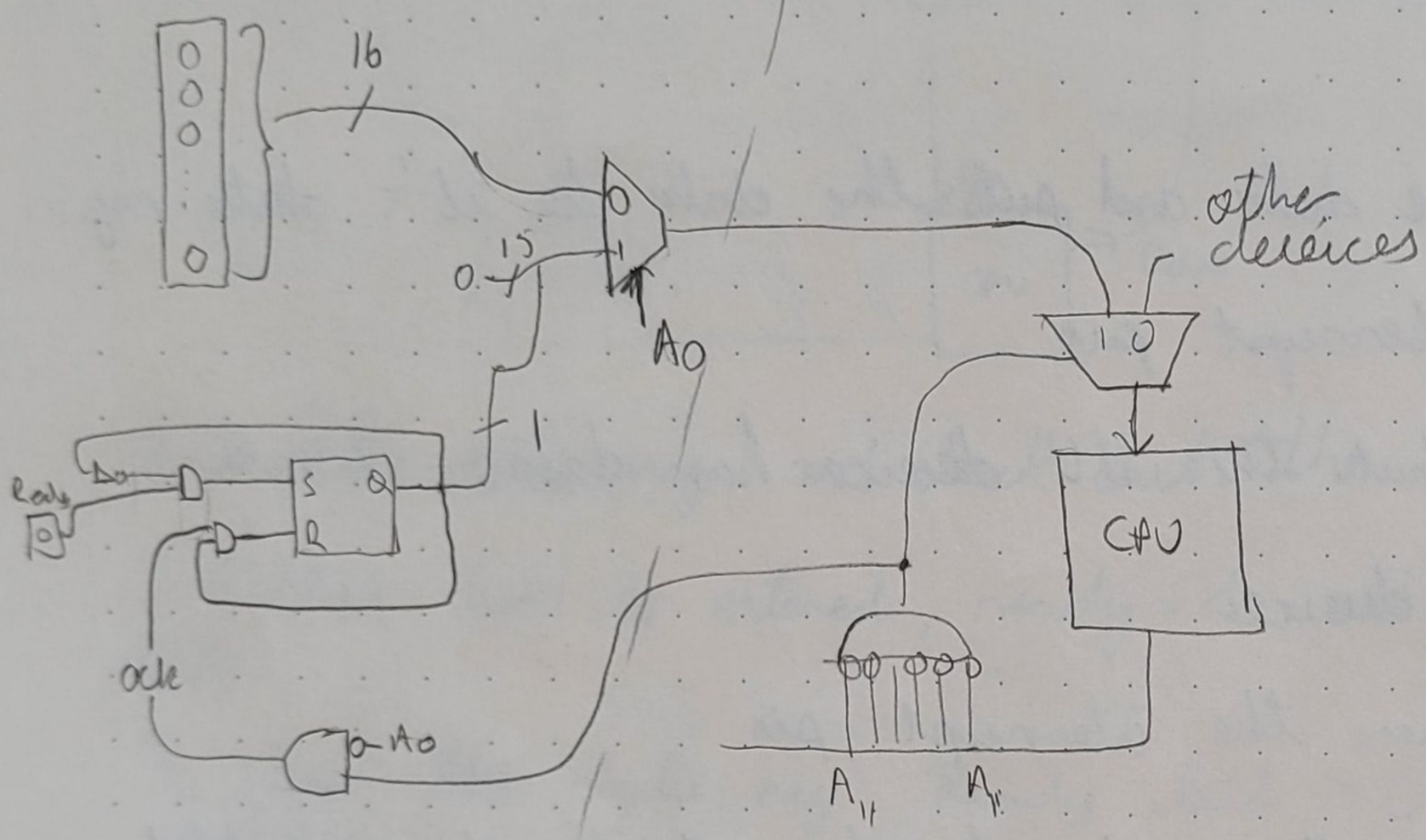


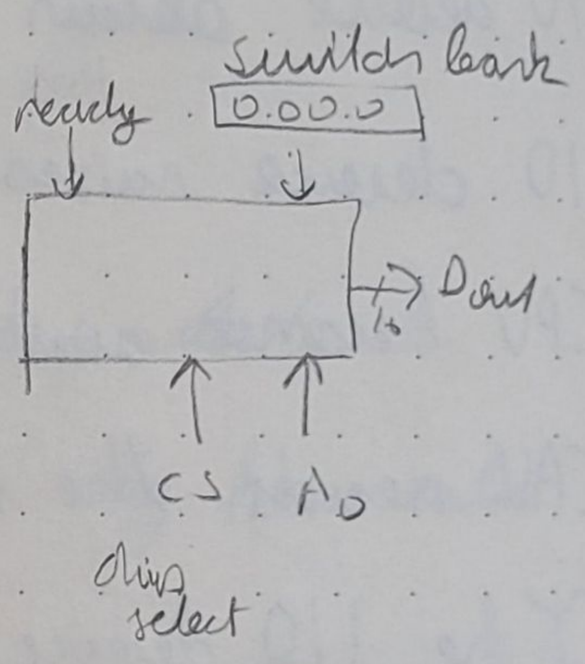
block int b

```
LDI, 0, 0xA12
LD   1, 0
LDI  2, 0xFB  → 1111000
AND  1, 1, 2
ST   0, 1
ST.1
```

26/12/2022

- Polling





keypad

09/01/2023

Programming for Interrupts

- 2 devices
- Device 1 has IRQ 2 and its ISR is at x0B1 and its data register is at 0x A00 (input dev.)
- Device 2 has IRQ 4 and ISR is at x0d4 and its data register is at 0x AC0 (input device)
- Monitor at 0x D00 (output device)
- Set stack at 0x E00

```
ldi  7, 0xE00   // stack set
ldi  0, 0x7F2   // idt addr for IRQ 2   (x7F0 IDT beginning)
ldi  1, 0x0B1   isr1
st   0, 1       // set up entry 2 in IDT
ldi  0, 0x7f4
ldi  1, 0x0d4
st   0, 1
sti             // open interrupts
jmp  loop       ----> polling here
isr1 ldi  2, 0xA00
     ld   2, 2
     ldi  3, 0xD00
     st   3, 2
     sti
     iret       // open interrupts again
isr2 ldi  4, 0xAC0
     ld   4, 4
     ldi  5, 0xD00
     st   5, 4
     sti
     iret
```

Example polling

```
loop: ldi  0  0xA01   } check status
      ld   1  0       }
      ldi  2  0x01    }
      and  1  1  2    }
      j2   loop1
      ldi  0  0xA00   } read data
      ld   2  0       }
      ldi  1  0xB00   } display
      st   1  4       }
```

12|.01|2023

